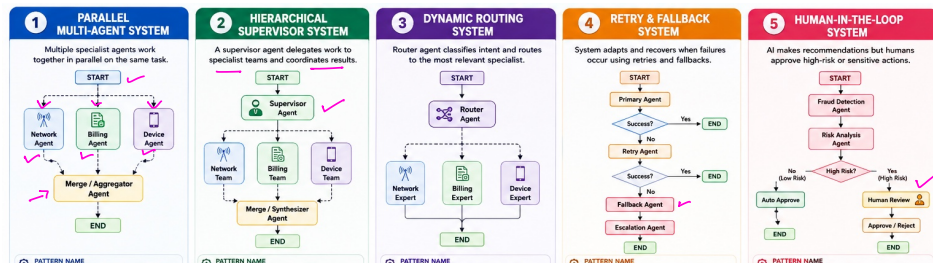


Topics For Today

- ① Different Patterns of Agents
- ② OpenAI Agent SDK
- ③ CrewAI
- ④ MCP

Agent → (x)or Kflow

- Parallel
- Hierarchical
- Retry & HLT



OpenAI Agent SDK

L> High level

- ① Agent
- ② Runnable
- ③ Tools
- ④ Handoff
- ⑤ Context
- ⑥ Memory
- ⑦ Guardrails
- ⑧ Output types

	Agent ✓ An LLM-powered entity with instructions, capabilities, and tools to accomplish tasks.
	Tools ✓ Functions or external services agents can call to interact with the real world.
	Memory ✓ Stores contextual information and past interactions to give agents continuity.
	Handoffs ✓ Allow agents to delegate tasks to other agents when appropriate.
	Guardrails ✓ Policies and checks that ensure safe, reliable, and compliant agent behavior.
	Sessions & State ✓ Manage conversation state, context, and agent execution state across runs.
	Multi-Agent Systems ✓ Coordinate multiple agents working together to solve complex problems.

① Agent → Brain →

Parameter	Purpose
✓ name	Identity of the agent (used in logs, handoffs, debugging)
instructions	System prompt that defines behavior and constraints ✓
(model)	The underlying LLM the agent runs on
✓ tools	Functions and external tools the agent can invoke
✓ handoffs	Other agents this agent can delegate work to
✓ memory	<u>Short-term or persistent conversation memory</u>
✓ output_type	Optional <u>structured-output</u> schema (Pydantic model)
✓ guardrails	<u>Validation and safety checks</u> applied to outputs

② Runner → Execution, Starting Workflow

Method	Purpose
✓ run_sync()	Synchronous execution — simplest, blocking call ✓
✓ run()	Asynchronous execution — for APIs and parallel workflows
✓ stream()	<u>Token-level streaming</u> for real-time output
✓ run_streamed()	Streams the entire agent workflow incrementally

③ Tools → Extending Capabilities

Python function

@tool → langchain

→ @function.tool → open ai agents

④ Handoffs → Delegation

Edge



⑤ Memory

→

Type	Purpose
Short-term ✓	Holds the current conversation turn-by-turn ✓
Long-term ✓	Persistent facts about the user or domain across sessions ✓
Session memory ✓	Workflow-scoped context shared across agents in one run ⊖

⑦ Context → Metadata, Runtime

⑧ Guardrails

→ Validation Safety

- input Guardrail
- Output Guardrail

⑨ Output types

— Agent → Specific

→

Utility	Purpose
Agent	Core AI worker
Runner	Executes workflows and drives the runtime
function_tool	Decorator that turns a Python function into a callable tool ✓
handoffs	Configures which agents can delegate to which ✓
memory	Persistence layer for conversation and state
guardrails	Validation and safety checks
output_type	Structured-output schemas (Pydantic)

CrewAI



Manager
(Crew)
Team members



Agents
A1-T1
A2-T2

Tasks

Crew →

manager

LangGraph vs OpenAI Agents SDK vs CrewAI

Compare the leading frameworks for building AI agents and multi-agent systems

OVERVIEW	LANGGRAPH	OPENAI AGENTS SDK	CREWAI
	Graph-based orchestration framework from the LangChain ecosystem. Design workflows as stateful graphs for reliable, complex agent systems.	OpenAI's lightweight framework for building agents with tools, handoffs, guardrails and memory. Optimized for OpenAI models.	Framework for building collaborative AI teams. Agents have roles, goals and work together like a team to get tasks done.
CORE IDEA	Graph-based orchestration	Agent-first (tools, handoffs, guardrails)	Team-first (roles, collaboration, delegation)
BEST FOR	Complex, production-grade workflows with state, branching and reliability	Fast development of GPT-native agents and tool-using workflows	Multi-agent collaboration and role-based team workflows
ARCHITECTURE (MENTAL MODEL)	Graph of nodes and edges with state	Linear / branching handoffs between agents	Team of agents with roles and goals
STRENGTHS	✓ Durable & reliable execution ✓ Excellent state management & checkpointing ✓ Human-in-the-loop & approvals ✓ Advanced observability & debugging ✓ Fine-grained control over workflows	✓ Extremely easy to start ✓ Minimal code, fast iteration ✓ Native tool calling & guardrails ✓ Great developer experience ✓ Best for rapid prototyping	✓ Very intuitive & easy to use ✓ Role-based collaboration ✓ Delegation & task management ✓ Fast demos & hackathons ✓ Human-like team workflow
WEAKNESSES	✗ More boilerplate ✗ Steeper learning curve ✗ Slower for simple prototypes	✗ Vendor lock-in (OpenAI) ✗ Less control over complex workflows ✗ Limited long-running state management	✗ Less deterministic ✗ Harder to debug at scale ✗ Can become messy in large systems
MODEL SUPPORT	Any LLM (OpenAI, Anthropic, Llama, Mistral, local models, etc.)	Primarily OpenAI models (GPT-4o, GPT-4, GPT-3.5, etc.)	Any LLM (OpenAI, Anthropic, Llama, Mistral, etc.)
LEARNING CURVE	Medium - High	Easy	Easy - Medium
IDEAL USE CASES	Enterprise Workflows, RAG & Research Pipelines, Finance & Insurance, Approval Systems, Autonomous Agents	AI Assistants, Customer Support, Coding Agents, Research Assistants, Rapid Prototypes	Content Creation, Marketing Automation, Research Teams, Education & Training, Creative Pipelines
EXAMPLE (MINI CODE)	<pre>graph.add_node("research", research_agent) graph.add_node("writer", writer_agent) graph.add_edge("research", "writer")</pre>	<pre>agent = Agent(name="Researcher", tools=[web_search, file_search])</pre>	<pre>researcher = Agent(role="Researcher") writer = Agent(role="Writer") crew = Crew([researcher, writer])</pre>
CORE DIFFERENCE	LANGGRAPH = WORKFLOW-FIRST You design the execution flow, state transitions, branching, and recovery.	OPENAI AGENTS SDK = AGENT-FIRST You design agents, tools, and handoffs. Best for simplicity and speed.	CREWAI = TEAM-FIRST You design roles, collaboration, and delegation. Best for teamwork.
			WHICH ONE SHOULD YOU CHOOSE? • Need reliability & complex workflows? → LangGraph • Fastest development with OpenAI? → Agents SDK • Want collaborative AI teams quickly? → CrewAI